

Projektrapport

Programmering i Typescript

Författare: Rune Granath

MITTUNIVERSITETET

Institutionen för Informations-system och teknologi

Författare: Rune Granath, rugr2500@student.miun.se

Utbildningsprogram: TWEUG, Webbutveckling, 120 hp

Huvudområde: Datateknik

Termin, år: HT, 2025

1 Sammanfattning

Denna rapport beskriver utvecklingen av en webbapplikation skapad som ett avslutande projekt i kursen Programmering i Typescript. Appens tema ska behandla ett fiktivt universitets hemsida där användare ska kunna gå igenom en stor extern JSON-fil med olika kurser. Dessa kurser ska kunna filtreras och sorteras på olika sätt och det ska finnas funktionalitet att skapa ett eget handplockat ramschema. Sidan ska byggas med ramverket Angular i programmeringsspråket TypeScript och behandla saker som att göra Http-anrop, skapa signaler, designa ett enhetligt utseende och data binding med hjälp av Angulars inbyggda funktionaliteter.

2 Innehållsförteckning

1	Sammanfattning	ii
2	Förord	iii
3	Innehållsförteckning	iv
4	Terminologi	v
5	Introduktion	1
5.1	Bakgrund och problemmotivering	1
5.2	Avgränsningar	1
5.3	Detaljerad problemformulering.....	1
5.4	Författarens bidrag	1
6	Teori.....	2
7	Metod.....	3
8	Konstruktion	4
9	Resultat	5
10	Slutsatser	6

3 Terminologi

Property Binding: Kopplar HTML-egenskaper till signaler med `[value]="variabelNamn()`

Event Binding: Kopplar händelser till metoder med `(click)="minMetod()`, datan flödar i motsatt riktning jämfört med **Property Binding**.

String Interpolation: Dubbla måsvingar som används för att skriva ut text eller variabler från koden på skärmen

Constructor: En fördefinierad metod som körs en gång vid skapelsen av en TypeScript-klass.

Ternary operator (villkorsstyrt uttryck): Fungerar som en kompakt if/else-sats som utvärderar ett villkor och returnerar ett av två värden baserat på om villkoret är sant eller falskt.

4 Introduktion

4.1 *Bakgrund och problemmotivering*

Det fiktiva universitetet Projektuniversitetet behöver ett sätt att hjälpa studenter som möts av en stor mängd kurser på samma gång med sammansättningen av ett ramschema där denna app ska lösa problemet med filtrering, validering och automatisering av denna process. Resultatet ska bli en blixtnabb skräddarsydd tjänst med ett proffsigt gränssnitt och designval.

4.2 *Detaljerad problemformulering*

Projektets huvudmål är att leverera en Single Page Application(SPA) som:

- Hämtar data asynkront via **HttpClient**.
- Möjliggör sökning och sortering i realtid.
- Hanterar ett personligt ramschema som sparas i webbläsarens minne.
- Säkerställer med validering att inga dubletter kan läggas till i schemat.
- Har ett proffsigt utseende som kan jämföras med en riktig studentportal

5 Teori

Olika verktyg som använts för att nå de uppsatta målen -

Angular och komponenter:

Angular är ett komponentbaserat ramverk utvecklat av Google. En komponent du genererar består av en TypeScript-klass för logik, en HTML-mall för struktur och en CSS-fil för stil. Genom att dela upp webbplatsen i olika delar på det viset blir kodbasen modulär och det blir enklare att skala arbetet med en mikrobit i taget eller direkt övergripande på makronivå. Angular använder även arbetssättet **SPA** (Single page application) där webbappen utgår från en huvudsaklig HTML-sida där innehållet uppdateras dynamiskt vilket bidrar till att ta bort fulla sidor-omladdning och snabbare hastighet. [1]

RxJs Observables:

Angulars redskap för att hantera asynkrona anrop och händelser. Genom att prenumerera på en **Observable** kan appen lyssna på strömmande data över tid och hantera anropen genom att reagera omedelbart när nya värden eller händelser inträffar. [2]

Signals:

Angular Signals har använts för att hantera tillstånd. Det går ut på att optimera renderingen genom att reaktivt endast uppdatera de delar av DOM:en som ändrats när datan uppdateras. Den kan vara **computed** vilket innebär att den automatiskt räknas ut och uppdateras baserat på andra signaler.

HttpClient:

En klient i Angular som används för att skicka nätverksanrop

Services:

Tjänster i Angular, fungerar som en motor för att organisera, återanvända och dela logik eller data mellan olika komponenter. De hjälper till att hålla komponenterna rena genom att flytta till exempel API-anrop och tillståndshantering ut från vyn.

Statisk typning med TypeScript:

Fungerar till skillnad från JavaScripts dynamiska typning med definiering av de datatyper som används i variabler och funktioner. Hjälper utvecklare genom konstant feedback med felhantering och autocompletion. [3]

LocalStorage:

Webbläsarens egna databas/minne som endast kan lagra data i strängformat. [4]

Interfaces:

Genom att använda så kallade interfaces kan man definiera kontraktet för ett objekt hur det ska vara utformat. TypeScript kommer därefter säga till om det är

något som saknas och ge förslag med kodkomplettering.

Angular material:

Ett UI-komponentsbibliotek som innehåller färdiga, moderna och responsiva komponenter vilka är redo att fogas in.

6 Metod

Följande metoder har använts för att uppnå de uppsatta målen:

Design:

För att få den rätt tänkta stilen på designen användes Angular material komponentbibliotek för att få ett proffsigt grundutseende med inbyggd responsivitet att utgå ifrån när det gäller saker som knappar, snackbars eller ikoner.

Importer i Angular:

Eftersom Angular är modulärt och du jobbar med en bit i taget måste varje sådan som sedan ska kommunicera med andra komponenter importeras i respektive fil där de används.

Sökning och filtrering:

För att hantera sökning och filtreringen användes först servicen **CourseService** där **HttpClient** injiceras till en variabel. Genom att sedan använda **RxJs Observables** hämtas data från uppgiftens JSON-fil asynkront. För att säkerställa att datan hanteras korrekt användes interfacet **Course** där koden vilket ger koden en specifik struktur att förhålla sig till.

I vyn när en användare sedan skriver i sökfältet/väljer i drop down-menyn för kurserna används **Event binding** som gör att värdet inuti fälten skickas till en reaktiv **Signal** och uppdaterar kurserna på det sätt som definierats i metoder för kurs-klassen.

Tillägg i ramschemat:

För att sedan kunna lägga till de kurser man väljer börjar det i komponentens HTML där en knapp blir kopplad till händelser (klick i detta fall). Med TypeScript skickas det interface-validerade objektet sedan som ett argument till en funktion **addCourse**. Med hjälp av servicens signal letas sedan listan igenom efter eventuella dubletter för att i så fall varna. Om kursen är unik uppdateras en signal som håller reda på alla valda kurser.

När listan i koden har uppdaterats anropas **LocalStorage** där objekten innan lagring omvandlas till textsträngar med **JSON.stringify()**. Via signaler uppdateras gränssnittet för ramschema sedan automatiskt och en **computed** signal sköter förändringen som summerar och visar de poäng alla sammanlagda kurser har. Om sidan sedan startas om med sparade kurser så kör servicen för ramschema sin **constructor** och omvandlar strängarna till JSON-objekt igen.

7 Konstruktion

Arbetet började med en planeringsfas för själva utformningen av appen där en exempelbild för uppgiften fick vara inspiration till designen. Även valet av **Angular schematic** komponentbibliotek verkade passa till ett utseende som ska vara tillgängligt och stilrent, i projektet användes detta huvudsakligen för knappar och symboler. Sedan kapslades CSS:en in för varje tillhörande komponent i deras respektive styling-filer.

Själva bygget startades sedan med att installera Angular och genereringen av alla undersidor, services och komponenter som behövdes med **'ng generate component/service x'**. För att Angular ska veta vilken komponent som ska visas på vilken URL definieras dessa inställningar i huvudskalets `app.routes.ts` fil. Här ställdes även sökvägar in för tomma URL:s som dirigerar tillbaka till index-sidan och en "404"-sida där en användare försöker skriva in någon plats som inte finns. Med hjälp av **RouterLink** som är Angulars inbyggda routingsystem för SPA görs det i HTML med `<a routerLink="länk">` istället för href, och för att styla aktiva sidolänkar används och importeras `routerLinkActive`.

Konstruktionen följde devisen att börja arbeta med datan innan den knyts ihop till varandra eller skrivs ut i vyer. Därför var det viktigt att få struktur med ett interface för ramschema-filen där varje fält typades ut som datatypen skulle tillhöra.

Två olika services skapades för att hantera kommunikationen mellan API-datan, en för inläsning av kurserna och en för att skapa ramschema. I servicen för kurserna injicerades Angulars **HttpClient**, som hanterar den asynkrona förfrågan om datan. Metoden `getCourses()` hämtar sedan datan och returnerar en observable (som representerar arrayen med kurser), vilket komponenterna sedan kan prenumerera på.

Ramschema-servicen utformades genom att följa valda kurser – `selectedCourses` vilken följer interfacet med en signal och initieras som en tom array. Den är `private` för att förhindra att komponenter utanför servicen kan manipulera datan direkt.

Med hjälp av `schedule = this.selectedCourses`; skapas en global version av schemat som komponenter kan läsa från, men inte ändra i direkt.

För att kunna visa den totala poängen av valda kurser skapas **totalPoints** vilket är en beräknad signal som bevakar de utvalda kurserna och räknar ut det totala poängantalet med JavaScripts `.reduce()` på ett reaktivt sätt.

En constructor skapas för att hantera saker som händer på sidan *en* gång, vilket i det här fallet gäller **localStorages** sparning av schemat.

En metod för att lägga till kurser skapas där `.some()` används för att kontrollera om en kurskod som läggs till redan finns i schemat. Om det är fallet varnar sidan med en snackbar-alert och annars när allt går bra så uppdateras `selectedCourses` signal med unika kurser, och `saveToStorage()` körs för att spara strängvärden i webbläsarminnet.

Slutligen behövs en metod för borttagning av kurser `removeFromSchedule` där en uppdatering sker utefter den förra `selectedCourses`-listan och skapar en ny

lista där alla kurser finns med utom den vi klickat 'ta bort' i gränssnittet på.

När services är på plats är det dags att lägga in funktionalitet för sortering och filtrering, vilket görs bäst i den tillhörande komponentens TypeScript-fil. Där börjar man med att importera alla funktionaliteter från angular, moduler, services och interfacet som behövs.

För kurssidan skapades sedan ett antal signaler för att hålla reda på komponentens tillstånd (state):

courselist: håller ursprungslistan av alla kurser.

Error: Håller ett felmeddelande om API-anropet skulle misslyckas.

FilterText: Håller den textsträng som användaren skriver i sökrutan

SortKey: Håller reda på vilken kolumn/egenskap i kurs-listan som sorteringen går efter, här definieras `courseCode` som är en unik kurskod och med `<keyof Course>` säger den att vi ska följa interfacet.

SortOrder: Håller reda på om sortering ska vara stigande eller fallande.

SelectedSubject: Håller det valda ämnet om användaren väljer ett genom dropdown-menyn.

Subjects: En beräknad signal som tar bort dubletter i vyn för ämneslistan.

Själva filtreringen sköts sedan med den beräknade **filteredCourses** som reagerar på **filterText**, **sortKey** och **sortOrder**. Den baseras på söktext som användaren skriver in och filtrerar den mot kurskod, kursnamn, ämne och progression, vilka görs om till gemener via **toLocaleLowerCase()** för att inte vara skiftes känslig. Slutligen sorteras det filtrerade resultatet genom att returnera en jämförelse via **localeCompare()**-funktionen. För att kunna vända listan om användaren vill ha omvänd ordning så används ett villkorsstyrt uttryck som gånger med `-1` för att kunna byta riktning.

Vidare byggs TypeScriptfilen för kurssidan på genom att få tillgång till metoderna som services erbjuder, vilka injiceras in.

Metoden **ngOnInit()** körs automatiskt när komponenten startas och anropar **loadCourses** som är metoden där vi prenumererar på **getCourses()** från kurs-servicen. Eftersom **HttpClient** jobbar med asynkrona strömmar bestämmer vi med **'next:'** att kursernas sparas vid lyckat anrop i signalen **courselist** och vid **'error:'** så fångas felmeddelandet upp och konsolloggas samt definieras som ett felmeddelande vilket kan skrivas ut i gränssnittet.

TypeScriptfilen för vyn som hanterar ramschemat blir relativt simpelt konstruerad när dess enda uppgift är att hämta data från schedule-servicen och skicka vidare användares klick för borttagningen av kurser tillbaka till servicen. Här injiceras Schedule som är satt till public då den ska vara åtkomlig för HTML, och metoden **remove** tar emot en kurskod från HTML-mallen när någon trycker på en knapp och talar om för servicen att radera den kursen. Lokala referenser/ genvägar skapas för att läsa av värdena från **scheduleServicens** kurser och totalpoäng och slippa skriva hela sökvägen i

Arbetet fortsatte sedan med att skriva ut data till vyerna. I kurser.html binds inputrutan för sökning ihop med signalen **filterText()** och varje gång användaren trycker på en tangent triggas input och tar texten från sökrutan och skickar den in i signalen. För listan med ämnen triggas inputen varje gång användaren byter det.

Med Angulars **@for** loopas sedan kurserna ut till rullgardinsmenyn via den validerade och sorterade signalen **subjects()** medan **'track' subject** säger att den textsträngen är unik. Med **property binding** och **string interpolation** bestäms sedan hur varje varv i loopen skapar den HTML som skrivs ut. Med **Event binding** kopplas **setSort**-metoden som triggar **filteredCourses** och de olika egenskaper som ska sorteras sedan till knappar.

För att skriva ut det totala antalet kurser som visas används **string interpolation** och **.length**, med en kombination av **ternär operator** vilken kollar om längden på **filteredCourses** är exakt 1, och ändrar då grammatiken i gränssnittet från 'kurs' till 'kurser'.

En till **@for**-loop som använder **'track'** på **\$index** skriver sedan ut kort med kursinfon med **string interpolation**. Hanteringen av en tom lista bestäms med **"@empty"** och bestäms då att **error()** ska skrivas ut till vyn. Slutligen med **@else** hanteras om allt är normalt men listan ändå är tom så visas ett meddelande om att kurshämtning pågår.

Datautskriften till ramschema-sidan fungerade på ett liknande sätt där totalpoäng och kursernas antal skrivs ut med string interpolation för refereringarna till **selectedCourses()** och en loop som följer **courseCode**. En knapp som använder **remove()**-metoden tar bort kurser via samma kod.

8 Resultat

Projektets huvudsakliga mål fullföljdes för de olika uppgifterna genom:

Hämtningen av data: Sköttes asynkront via injektion av HttpClient i en service.

För sökning och filtrering: Stigande och fallande bokstavssortering i realtid, möjlig sortering på ämne och filtrering av användarens inputtext.

Hantering av personligt ramschema: Användaren kan skraddarsy sina egna kurser som visas på en undersida där de sparas i webbläsarens minne. Kursernas totala poäng visas dynamiskt.

Validering: Validering sker via en dubblettkoll och förhindrar att samma kurser läggs till mer än en gång i schemat.

Utseende: Följde efterfrågad exempelgrundstil med några tillägg och användandet av komponentbibliotek till hjälp. Totala antalet kurser skrivs ut till skärmen.

9 Slutsatser

Arbetet med att utveckla ramschema-applikationen har varit mycket lärorikt, och slutprodukten uppfyller alla de mål jag satte upp i projektplanen. Arbetet med Angular som ramverk hade en ordentlig inkörsport med nya syntaxer, importter och metoder som för det mesta som var lite annorlunda men ändå påminde om tidigare lärdomar i kurserna. Men när du väl tagit dig igenom den var det enkelt att se hur Angular kan hjälpa utvecklingen på många sätt. Främst då genom betydlig nedkortning av koden men också att jobba med inkapslade delar av jobbet för bättre kontroll på detalj- och makronivå. Att använda TypeScript istället för uteslutande JavaScript var också en liknande upplevelse, men med en betydligt mer logisk övergång och medförde mycket struktur till kodandet.

Annars gick arbetet bra, om det är något som hade kunnat byggts vidare på om ytterligare tid spenderats vore det en utveckling av användarvänligheten i form av till exempel en AI-chattbot eller landningssida som index.

Källförteckning

- [1] Google. Angular Documentation [Internet]. Mountain View: Google; 2026 [hämtad 2026-05-19]. Tillgänglig från: <https://angular.dev/>
- [2] ReactiveX. RxJS Docs [Internet]. [ort okänd]: ReactiveX; 2026 [hämtad 2026-05-19]. Tillgänglig från: <https://rxjs.dev/>
- [3] Microsoft. TypeScript Documentation [Internet]. Redmond: Microsoft; 2026 [hämtad 2026-05-19]. Tillgänglig från: <https://www.typescriptlang.org/docs/>
- [4] Mozilla Developer Network (MDN). Window: localStorage [Internet]. Mozilla; 2026 [hämtad 2026-05-19]. Tillgänglig från: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>